

Practice Assignment

Transactions · Joins · Indexing · Query Optimization

Part – 1

```
CREATE TABLE customers (  
  id INT PRIMARY KEY AUTO_INCREMENT,  
  name VARCHAR(100),  
  email VARCHAR(100) UNIQUE,  
  signup_date DATE  
);
```

```
CREATE TABLE restaurants (  
  id INT PRIMARY KEY AUTO_INCREMENT,  
  name VARCHAR(100),  
  cuisine VARCHAR(50)  
);
```

```
CREATE TABLE orders (  
  id INT PRIMARY KEY AUTO_INCREMENT,  
  customer_id INT,  
  restaurant_id INT,  
  total DECIMAL(10,2),  
  status VARCHAR(20),  
  created_at DATETIME,  
  
  FOREIGN KEY (customer_id) REFERENCES customers(id),  
  FOREIGN KEY (restaurant_id) REFERENCES restaurants(id)  
);
```

```
CREATE TABLE accounts (  
  id INT PRIMARY KEY AUTO_INCREMENT,  
  name VARCHAR(100),
```

```
    balance DECIMAL(10,2)
);
```

```
INSERT INTO customers (name, email, signup_date) VALUES
('Padmraj Parmar', 'padmraj.parmar@gmail.com', '2026-01-05'),
('Aditi Sharma', 'aditi.sharma@gmail.com', '2026-01-08'),
('Rohan Patel', 'rohan.patel@gmail.com', '2026-01-12'),
('Priyanshi Joshi', 'priyanshi.joshi@gmail.com', '2026-01-18'),
('Dhruv Shah', 'dhruv.shah@gmail.com', '2026-01-25'),
('Khushi Mehta', 'khushi.mehta@gmail.com', '2026-02-02'),
('Parth Desai', 'parth.desai@gmail.com', '2026-02-08'),
('Sneha Nair', 'sneha.nair@gmail.com', '2026-02-14'),
('Harsh Modi', 'harsh.modi@gmail.com', '2026-02-20'),
('Nidhi Kapoor', 'nidhi.kapoor@gmail.com', '2026-02-28'),
('Karan Trivedi', 'karan.trivedi@gmail.com', '2026-03-05'),
('Isha Gupta', 'isha.gupta@gmail.com', '2026-03-12'),
('Rahul Soni', 'rahul.soni@gmail.com', '2026-03-20'),
('Neha Singh', 'neha.singh@gmail.com', '2026-03-28'),
('Vikas Chauhan', 'vikas.chauhan@gmail.com', '2026-04-05');
```

```
INSERT INTO restaurants (name, cuisine) VALUES
('Royal Spice', 'Indian'),
('Italian Oven', 'Italian'),
('Beijing Bowl', 'Chinese'),
('Burger Factory', 'American'),
('Mexican Fiesta', 'Mexican'),
('Tokyo Express', 'Japanese'),
('Punjabi Dhaba', 'North Indian'),
```

('Cafe Delight', 'Continental');

```
INSERT INTO orders (customer_id, restaurant_id, total, status, created_at) VALUES
(1,1,540.00,'completed','2026-01-06 12:30:00'),
(1,3,320.00,'completed','2026-01-18 19:20:00'),
(1,6,890.00,'pending','2026-02-14 18:00:00'),
(2,2,760.00,'completed','2026-01-10 13:15:00'),
(2,5,410.00,'completed','2026-02-01 20:10:00'),
(2,8,290.00,'cancelled','2026-03-05 12:40:00'),
(3,4,650.00,'completed','2026-01-15 19:30:00'),
(3,1,430.00,'completed','2026-02-12 12:10:00'),
(3,7,980.00,'completed','2026-03-18 20:00:00'),
(4,6,1100.00,'completed','2026-01-20 18:45:00'),
(4,3,470.00,'pending','2026-02-20 13:20:00'),
(4,2,620.00,'completed','2026-03-25 19:15:00'),
(5,5,360.00,'completed','2026-01-28 12:30:00'),
(5,8,820.00,'completed','2026-02-26 18:50:00'),
(5,4,510.00,'completed','2026-03-30 20:30:00'),
(6,7,720.00,'completed','2026-02-05 13:10:00'),
(6,1,580.00,'completed','2026-03-02 19:00:00'),
(6,6,930.00,'pending','2026-04-04 18:40:00'),
(7,2,680.00,'completed','2026-02-10 12:45:00'),
(7,5,440.00,'completed','2026-03-08 20:00:00'),
(7,3,390.00,'cancelled','2026-04-08 13:00:00'),
(8,4,560.00,'completed','2026-02-18 19:40:00'),
(8,7,1250.00,'completed','2026-03-12 21:00:00'),
(8,1,470.00,'completed','2026-04-10 12:20:00'),
(9,6,880.00,'completed','2026-02-24 18:15:00'),
(9,2,630.00,'pending','2026-03-16 20:45:00'),
(9,8,350.00,'completed','2026-04-12 13:30:00'),
(10,3,420.00,'completed','2026-03-01 12:50:00'),
(10,5,710.00,'completed','2026-03-20 18:30:00'),
(10,7,980.00,'completed','2026-04-14 19:20:00'),
(11,1,520.00,'completed','2026-03-06 13:40:00'),
(11,4,610.00,'pending','2026-03-28 20:15:00'),
```

```
(11,6,790.00,'completed','2026-04-16 18:00:00'),  
(12,8,450.00,'completed','2026-03-15 12:30:00'),  
(12,2,690.00,'completed','2026-04-01 19:10:00'),  
(12,5,380.00,'completed','2026-04-18 13:20:00');
```

```
INSERT INTO accounts (name, balance) VALUES  
('Padmraj', 2500.00),  
('Kiran', 1800.00);
```

Part – 2

TASK 1 Transactions (ACID)

a)

Schema SQL

```

1 CREATE TABLE customers (
2   id INT PRIMARY KEY AUTO_INCREMENT,
3   name VARCHAR(100),
4   email VARCHAR(100) UNIQUE,
5   signup_date DATE
6 );
7
8 CREATE TABLE restaurants (
9   id INT PRIMARY KEY AUTO_INCREMENT,
10  name VARCHAR(100),
11  cuisine VARCHAR(50)
12 );
13
14 CREATE TABLE orders (

```

Text to DDL

Query SQL

```

1 START TRANSACTION;
2
3 UPDATE accounts
4 SET balance = balance - 500
5 WHERE name = 'Padmraj';
6
7 UPDATE accounts
8 SET balance = balance + 500
9 WHERE name = 'Kiran';
10
11 ROLLBACK;
12
13 SELECT * FROM accounts;

```

Results
Copy as Markdown

Query #5 Execution time: 12.00ms

There are no results to be displayed.

Query #5 Execution time: 0.33ms

id	name	balance
1	Padmraj	2500.0
2	Kiran	1800.0

b)

Schema SQL

```

1 CREATE TABLE customers (
2   id INT PRIMARY KEY AUTO_INCREMENT,
3   name VARCHAR(100),
4   email VARCHAR(100) UNIQUE,
5   signup_date DATE
6 );
7
8 CREATE TABLE restaurants (
9   id INT PRIMARY KEY AUTO_INCREMENT,
10  name VARCHAR(100),
11  cuisine VARCHAR(50)
12 );
13
14 CREATE TABLE orders (

```

Text to DDL

Query SQL

```

1 START TRANSACTION;
2
3 UPDATE accounts
4 SET balance = balance - 500
5 WHERE name = 'Padmraj';
6
7 UPDATE accounts
8 SET balance = balance + 500
9 WHERE name = 'Kiran';
10
11 COMMIT;
12
13 SELECT * FROM accounts;

```

Results
Copy as Markdown

Query #5 Execution time: 0.03ms

There are no results to be displayed.

Query #5 Execution time: 0.16ms

id	name	balance
1	Padmraj	2000.0
2	Kiran	2300.0

c)

Atomicity is the ACID property that ensures the money is never lost during a transfer. It makes sure that either both operations (debit and credit) happen successfully, or neither happens. If any error occurs, the transaction is rolled back, so both account balances remain unchanged.

TASK 2 Joins

a)

Schema SQL

```

1 CREATE TABLE customers (
2   id INT PRIMARY KEY AUTO_INCREMENT,
3   name VARCHAR(100),
4   email VARCHAR(100) UNIQUE,
5   signup_date DATE
6 );
7
8 CREATE TABLE restaurants (
9   id INT PRIMARY KEY AUTO_INCREMENT,
10  name VARCHAR(100),
11  cuisine VARCHAR(50)
12 );
13
14 CREATE TABLE orders (

```

Query SQL

```

20 SET @customer_id = 100;
21 WHERE name = 'Kiran';
22
23 COMMIT;
24
25 SELECT * FROM accounts;
26
27 SELECT DISTINCT
28   c.id,
29   c.name,
30   c.email
31 FROM customers c
32 INNER JOIN orders o
33 ON c.id = o.customer_id;

```

Results

Query #10 Execution time: 0.31ms

id	name	email
1	Padmraj Parmar	padmraj.parmar@gmail.com
2	Aditi Sharma	aditi.sharma@gmail.com
3	Rohan Patel	rohan.patel@gmail.com
4	Priyanshi Joshi	priyanshi.joshi@gmail.com

b)

Schema SQL

```

39 ('Parth Desai', 'parth.desai@gmail.com', '2026-02-08'),
40 ('Sneha Nair', 'sneha.nair@gmail.com', '2026-02-14'),
41 ('Harsh Modi', 'harsh.modi@gmail.com', '2026-02-20'),
42 ('Nidhi Kapoor', 'nidhi.kapoor@gmail.com', '2026-02-28'),
43 ('Karan Trivedi', 'karan.trivedi@gmail.com', '2026-03-05'),
44 ('Isha Gupta', 'isha.gupta@gmail.com', '2026-03-12'),
45 ('Rahul Soni', 'rahul.soni@gmail.com', '2026-03-20'),
46 ('Neha Singh', 'neha.singh@gmail.com', '2026-03-28'),
47 ('Vikas Chauhan', 'vikas.chauhan@gmail.com', '2026-04-05');
48
49 INSERT INTO restaurants (name, cuisine) VALUES
50 ('Royal Spice', 'Indian'),
51 ('Italian Oven', 'Italian'),
52 ('Beijing Bowl', 'Chinese');

```

Query SQL

```

29 c.name,
30 c.email
31 FROM customers c
32 INNER JOIN orders o
33 ON c.id = o.customer_id;
34
35 SELECT
36   c.id,
37   c.name,
38   c.email
39 FROM customers c
40 LEFT JOIN orders o
41 ON c.id = o.customer_id
42 WHERE o.id IS NULL;

```

Results

Query #11 Execution time: 0.19ms

id	name	email
13	Rahul Soni	rahul.soni@gmail.com
14	Neha Singh	neha.singh@gmail.com
15	Vikas Chauhan	vikas.chauhan@gmail.com

c)

A marketing team prefers query (b) because it also shows customers who have never placed an order, so they can target them with special offers or promotions.

TASK 3 Indexing

a)

We are using the **status** column from the orders table, which currently does not have an index.

b)

Schema SQL

```

59
60 INSERT INTO orders (customer_id, restaurant_id, total, status, created_at) VALUES
61 (1,1,540.00,'completed','2026-01-06 12:30:00'),
62 (1,3,320.00,'completed','2026-01-18 19:20:00'),
63 (1,6,890.00,'pending','2026-02-14 18:00:00'),
64 (2,2,760.00,'completed','2026-01-10 13:15:00'),
65 (2,5,410.00,'completed','2026-02-01 20:10:00'),
66 (2,8,290.00,'cancelled','2026-03-05 12:40:00'),
67 (3,4,650.00,'completed','2026-01-15 19:30:00'),
68 (3,1,430.00,'completed','2026-02-12 12:10:00'),
69 (3,7,980.00,'completed','2026-03-18 20:00:00'),
70 (4,6,1100.00,'completed','2026-01-20 18:45:00'),
71 (4,3,470.00,'pending','2026-02-20 13:20:00'),
72 (4,2,620.00,'completed','2026-03-25 19:15:00').

```

Text to DDL

Query SQL

```

1
2 EXPLAIN ANALYZE
3 SELECT * FROM orders
4 WHERE status = 'pending';

```

Results
Copy as Markdown

Query #1 Execution time: 0.38ms

EXPLAIN

-> Filter: (orders.'status' = 'pending') (cost=3.85 rows=3.6) (actual time=0.0275..0.0575 rows=5 loops=1) -> Table scan on orders (cost=3.85 rows=36) (actual time=0.0189..0.0466 rows=36 loops=1)

c) & d)

Schema SQL

```

59
60 INSERT INTO orders (customer_id, restaurant_id, total, status, created_at) VALUES
61 (1,1,540.00,'completed','2026-01-06 12:30:00'),
62 (1,3,320.00,'completed','2026-01-18 19:20:00'),
63 (1,6,890.00,'pending','2026-02-14 18:00:00'),
64 (2,2,760.00,'completed','2026-01-10 13:15:00'),
65 (2,5,410.00,'completed','2026-02-01 20:10:00'),
66 (2,8,290.00,'cancelled','2026-03-05 12:40:00'),
67 (3,4,650.00,'completed','2026-01-15 19:30:00'),
68 (3,1,430.00,'completed','2026-02-12 12:10:00'),
69 (3,7,980.00,'completed','2026-03-18 20:00:00'),
70 (4,6,1100.00,'completed','2026-01-20 18:45:00'),
71 (4,3,470.00,'pending','2026-02-20 13:20:00'),
72 (4,2,620.00,'completed','2026-03-25 19:15:00').

```

Text to DDL

Query SQL

```

1 CREATE INDEX idx_orders_status ON orders(status);
2
3 EXPLAIN ANALYZE
4 SELECT * FROM orders
5 WHERE status = 'pending';

```

Results
Copy as Markdown

Query #1 Execution time: 9.6ms

There are no results to be displayed.

Query #2 Execution time: 0.43ms

EXPLAIN

-> Index lookup on orders using idx_orders_status (status = 'pending') (cost=1.25 rows=5) (actual time=0.0221..0.024 rows=5 loops=1)

e) Before adding the index, MySQL had to perform a full table scan examining all 36 rows; after indexing, it performed an instant index lookup examining only the 5 matching rows, significantly cutting down execution time.

TASK 4 Query Optimization

a)

Schema SQL

```

59
60 INSERT INTO orders (customer_id, restaurant_id, total, status, created_at) VALUES
61 (1,1,540.00,'completed','2026-01-06 12:30:00'),
62 (1,3,320.00,'completed','2026-01-18 19:20:00'),
63 (1,6,890.00,'pending','2026-02-14 18:00:00'),
64 (2,2,760.00,'completed','2026-01-10 13:15:00'),
65 (2,5,410.00,'completed','2026-02-01 20:10:00'),
66 (2,8,290.00,'cancelled','2026-03-05 12:40:00'),
67 (3,4,650.00,'completed','2026-01-15 19:30:00'),
68 (3,1,430.00,'completed','2026-02-12 12:10:00'),
69 (3,7,980.00,'completed','2026-03-18 20:00:00'),
70 (4,6,1100.00,'completed','2026-01-20 18:45:00'),
71 (4,3,470.00,'pending','2026-02-20 13:20:00'),
72 (4,2,620.00,'completed','2026-03-25 19:15:00').

```

Query SQL

```

1 SELECT o.*
2 FROM orders o
3 INNER JOIN customers c
4 ON o.customer_id = c.id
5 WHERE c.signup_date > '2026-02-01';

```

Results

Copy as Markdown

Query #1 Execution time: 0.42ms

id	customer_id	restaurant_id	total	status	created_at
16	6	7	720.0	completed	2026-02-05 13:10:00
17	6	1	580.0	completed	2026-03-02 19:00:00
18	6	6	930.0	pending	2026-04-04 18:40:00
19	7	2	680.0	completed	2026-02-10 12:45:00

b)

Run
Save
Load Example
Collaborate

Sign in
Have any feedback?

Schema SQL

```

56 ('Punjabi Dhaba', 'North Indian'),
57 ('Cafe Delight', 'Continental');
58
59
60 INSERT INTO orders (customer_id, restaurant_id, total, status, created_at) VALUES
61 (1,1,540.00,'completed','2026-01-06 12:30:00'),
62 (1,3,320.00,'completed','2026-01-18 19:20:00'),
63 (1,6,890.00,'pending','2026-02-14 18:00:00'),
64 (2,2,760.00,'completed','2026-01-10 13:15:00'),
65 (2,5,410.00,'completed','2026-02-01 20:10:00'),
66 (2,8,290.00,'cancelled','2026-03-05 12:40:00'),
67 (3,4,650.00,'completed','2026-01-15 19:30:00'),
68 (3,1,430.00,'completed','2026-02-12 12:10:00'),
69 (3,7,980.00,'completed','2026-03-18 20:00:00'),

```

Query SQL

```

1 SELECT
2   o.id,
3   o.restaurant_id,
4   o.status,
5   o.total
6 FROM orders o
7 INNER JOIN customers c
8 ON o.customer_id = c.id
9 WHERE c.signup_date > '2026-02-01';

```

Results

Copy as Markdown

Query #1 Execution time: 0.37ms

id	restaurant_id	status	total
16	7	completed	720.0
17	1	completed	580.0
18	6	pending	930.0
19	2	completed	680.0

c)

Schema SQL

```
56 ('Punjabi Dhaba', 'North Indian'),
57 ('Cafe Delight', 'Continental');
58
59
60 INSERT INTO orders (customer_id, restaurant_id, total, status, created_at) VALUES
61 (1,1,540.00,'completed','2026-01-06 12:30:00'),
62 (1,3,320.00,'completed','2026-01-18 19:20:00'),
63 (1,6,890.00,'pending','2026-02-14 18:00:00'),
64 (2,2,760.00,'completed','2026-01-10 13:15:00'),
65 (2,5,410.00,'completed','2026-02-01 20:10:00'),
66 (2,8,290.00,'cancelled','2026-03-05 12:40:00'),
67 (3,4,650.00,'completed','2026-01-15 19:30:00'),
68 (3,1,430.00,'completed','2026-02-12 12:10:00'),
69 (3,7,980.00,'completed','2026-03-18 20:00:00'),
```

Text to DDL

Query SQL

```
1 EXPLAIN ANALYZE
2 SELECT *FROM orders WHERE customer_id IN
3 (
4     SELECT id FROM customers
5     WHERE signup_date > '2026-02-01'
6 );
7
8 EXPLAIN ANALYZE
9 SELECT o.id, o.restaurant_id, o.status,o.total
10 FROM orders o
11 INNER JOIN customers c
12 ON o.customer_id = c.id
13 WHERE c.signup_date > '2026-02-01';
```

Results

Copy as Markdown

-> Nested loop inner join (cost=3.28 rows=4.67) (actual time=0.0403. 0.0833 rows=21 loops=1) -> Filter: (customers.signup_date > DATE'2026-02-01') (cost=1.65 rows=4.67) (actual time=0.0185. 0.0239 rows=10 loops=1) -> Table scan on customers (cost=1.65 rows=14) (actual time=0.0152. 0.0206 rows=15 loops=1) -> Index lookup on orders using customer_id (customer_id = customers.id) (cost=0.271 rows=1) (actual time=0.00457. 0.00514 rows=2.1 loops=10)

Query #2

Execution time: 0.3ms

EXPLAIN

-> Nested loop inner join (cost=3.28 rows=4.67) (actual time=0.02. 0.0607 rows=21 loops=1) -> Filter: (c.signup_date > DATE'2026-02-01') (cost=1.65 rows=4.67) (actual time=0.0102. 0.0151 rows=10 loops=1) -> Table scan on c (cost=1.65 rows=14) (actual time=0.00811. 0.0129 rows=15 loops=1) -> Index lookup on o using customer_id (customer_id = c.id) (cost=0.271 rows=1) (actual time=0.00367. 0.0042 rows=2.1 loops=10)

TASK 5 The Customer Profile Page

a)

Schema SQL

```
56 ('Punjabi Dhaba', 'North Indian'),
57 ('Cafe Delight', 'Continental');
58
59
60 INSERT INTO orders (customer_id, restaurant_id, total, status, created_at) VALUES
61 (1,1,540.00,'completed','2026-01-06 12:30:00'),
62 (1,3,320.00,'completed','2026-01-18 19:20:00'),
63 (1,6,890.00,'pending','2026-02-14 18:00:00'),
64 (2,2,760.00,'completed','2026-01-10 13:15:00'),
65 (2,5,410.00,'completed','2026-02-01 20:10:00'),
66 (2,8,290.00,'cancelled','2026-03-05 12:40:00'),
67 (3,4,650.00,'completed','2026-01-15 19:30:00'),
68 (3,1,430.00,'completed','2026-02-12 12:10:00'),
69 (3,7,980.00,'completed','2026-03-18 20:00:00'),
```

Text to DDL

Query SQL

```
1 SELECT
2     r.name AS restaurant_name,
3     o.total,
4     o.status,
5     o.created_at
6 FROM orders o
7 INNER JOIN restaurants r
8 ON o.restaurant_id = r.id
9 WHERE o.customer_id = 1
10 ORDER BY o.created_at DESC
11 LIMIT 5;
```

Results

Copy as Markdown

Query #1

Execution time: 0.33ms

restaurant_name	total	status	created_at
Tokyo Express	890.0	pending	2026-02-14 18:00:00
Beijing Bowl	320.0	completed	2026-01-18 19:20:00
Royal Spice	540.0	completed	2026-01-06 12:30:00

b)

Schema SQL

```
56 ('Punjabi Dhaba', 'North Indian'),
57 ('Cafe Delight', 'Continental');
58
59
60 INSERT INTO orders (customer_id, restaurant_id, total, status, created_at) VALUES
61 (1,1,540.00,'completed','2026-01-06 12:30:00'),
62 (1,3,320.00,'completed','2026-01-18 19:20:00'),
63 (1,6,890.00,'pending','2026-02-14 18:00:00'),
64 (2,2,760.00,'completed','2026-01-10 13:15:00'),
65 (2,5,410.00,'completed','2026-02-01 20:10:00'),
66 (2,8,290.00,'cancelled','2026-03-05 12:40:00'),
67 (3,4,650.00,'completed','2026-01-15 19:30:00'),
68 (3,1,430.00,'completed','2026-02-12 12:10:00'),
69 (3,7,980.00,'completed','2026-03-18 20:00:00'),
```

Text to DDL

Query SQL

```
1 EXPLAIN ANALYZE
2 SELECT
3   r.name AS restaurant_name,
4   o.total,
5   o.status,
6   o.created_at
7 FROM orders o
8 JOIN restaurants r
9 ON o.restaurant_id = r.id
10 WHERE o.customer_id = 1
11 ORDER BY o.created_at DESC
12 LIMIT 5;
```

Results

Copy as Markdown

Query #1 Execution time: 17.05ms

EXPLAIN

-> Limit: 5 row(s) (cost=2.1 rows=3) (actual time=0.0502..0.0546 rows=3 loops=1) -> Nested loop inner join (cost=2.1 rows=3) (actual time=0.0466..0.0507 rows=3 loops=1) -> Sort: o.created_at DESC (cost=1.05 rows=3) (actual time=0.0341..0.0346 rows=3 loops=1) -> Filter: (o.restaurant_id is not null) (cost=1.05 rows=3) (actual time=0.0177..0.0198 rows=3 loops=1) -> Index lookup on o using customer_id (customer_id = 1) (cost=1.05 rows=3) (actual time=0.0165..0.0183 rows=3 loops=1) -> Single-row index lookup on r using PRIMARY (id = o.restaurant_id) (cost=0.283 rows=1) (actual time=0.00468..0.00473 rows=1 loops=3)

c) & d)

Schema SQL

```
56 ('Punjabi Dhaba', 'North Indian'),
57 ('Cafe Delight', 'Continental');
58
59
60 INSERT INTO orders (customer_id, restaurant_id, total, status, created_at) VALUES
61 (1,1,540.00,'completed','2026-01-06 12:30:00'),
62 (1,3,320.00,'completed','2026-01-18 19:20:00'),
63 (1,6,890.00,'pending','2026-02-14 18:00:00'),
64 (2,2,760.00,'completed','2026-01-10 13:15:00'),
65 (2,5,410.00,'completed','2026-02-01 20:10:00'),
66 (2,8,290.00,'cancelled','2026-03-05 12:40:00'),
67 (3,4,650.00,'completed','2026-01-15 19:30:00'),
68 (3,1,430.00,'completed','2026-02-12 12:10:00'),
69 (3,7,980.00,'completed','2026-03-18 20:00:00'),
```

Text to DDL

Query SQL

```
1 CREATE INDEX idx_customer_created
2 ON orders(customer_id, created_at);
3
4 EXPLAIN ANALYZE
5 SELECT r.name AS restaurant_name,o.total,o.status,o.created_at
6 FROM orders o
7 JOIN restaurants r
8 ON o.restaurant_id = r.id
9 WHERE o.customer_id = 1
10 ORDER BY o.created_at DESC
11 LIMIT 5;
```

Results

Copy as Markdown

Query #1 Execution time: 18.68ms

There are no results to be displayed.

Query #2 Execution time: 0.65ms

EXPLAIN

-> Limit: 5 row(s) (cost=2.1 rows=3) (actual time=0.0434..0.0497 rows=3 loops=1) -> Nested loop inner join (cost=2.1 rows=3) (actual time=0.0394..0.0454 rows=3 loops=1) -> Filter: (o.restaurant_id is not null) (cost=1.05 rows=3) (actual time=0.0304..0.0329 rows=3 loops=1) -> Index lookup on o using idx_customer_created (customer_id = 1) (reverse) (cost=1.05 rows=3) (actual time=0.0291..0.0312 rows=3 loops=1) -> Single-row index lookup on r using PRIMARY (id = o.restaurant_id) (cost=0.283 rows=1) (actual time=0.00344..0.00348 rows=1 loops=3)

e)

Schema SQL

```

56 ('Punjabi Dhaba', 'North Indian'),
57 ('Cafe Delight', 'Continental');
58
59
60 INSERT INTO orders (customer_id, restaurant_id, total, status, created_at) VALUES
61 (1,1,540.00,'completed','2026-01-06 12:30:00'),
62 (1,3,320.00,'completed','2026-01-18 19:20:00'),
63 (1,6,890.00,'pending','2026-02-14 18:00:00'),
64 (2,2,760.00,'completed','2026-01-10 13:15:00'),
65 (2,5,410.00,'completed','2026-02-01 20:10:00'),
66 (2,8,290.00,'cancelled','2026-03-05 12:40:00'),
67 (3,4,650.00,'completed','2026-01-15 19:30:00'),
68 (3,1,430.00,'completed','2026-02-12 12:10:00'),
69 (3,7,980.00,'completed','2026-03-18 20:00:00'),

```

Text to DDL

Query SQL

```

1 SELECT
2     r.name AS restaurant_name,
3     o.total,
4     o.status,
5     o.created_at
6 FROM orders o
7 INNER JOIN restaurants r
8 ON o.restaurant_id = r.id
9 WHERE o.customer_id = 1
10 ORDER BY o.created_at DESC
11 LIMIT 5;

```

Copy as Markdown

Results

Query #1 Execution time: 0.35ms

restaurant_name	total	status	created_at
Tokyo Express	890.0	pending	2026-02-14 18:00:00
Beijing Bowl	320.0	completed	2026-01-18 19:20:00
Royal Spice	540.0	completed	2026-01-06 12:30:00

f)

We have created an index on (customer_id, created_at) because the query filters data using customer_id and sorts it using created_at. The index helps MySQL find the required records faster without scanning unnecessary data. As a result, the execution time decreased from 17.05 ms to 0.65 ms, making the query more efficient.